DevTrack — Complete Technology Stack & Industry Guide

A full reference for **what DevTrack uses**, **why**, **alternatives**, **how to make it efficient**, **what each technology's job is in the industry today**, and **what real companies do with the same ideas**.

Table of contents

1. Product context
2. Stack at a glance
3. Architecture overview
4. Language & UI layer (deep dive)
5. Data & backend (deep dive)
6. Auth, validation, security
7. Charts, motion, design system
8. DevTrack domain systems
9. Tooling, CI, quality
10. Optional services
11. Key role of each technology today
12. What companies are doing with this tech
13. Company role map (who uses what for what)
14. Efficiency playbook (current + roadmap)
15. When to keep vs replace a choice
16. Interview / portfolio talking points
17. Related docs
18. Appendices

---

1. Product context

DevTrack is a multi-platform DSA (Data Structures & Algorithms) progress tracker.

Core jobs

- Link coding profiles (LeetCode, Codeforces, AtCoder, HackerRank, CodeChef)
- Sync submissions / solves into one database
- Show heatmaps, streaks, stats, recent activity
- Filter/compare platforms on the Overview dashboard
- Aggregate upcoming contests
- Support teams, public profiles, auth

Why the stack matters
This product is an **integration + dashboard** app: most complexity is syncing unreliable third-party APIs and presenting personalized analytics. The stack must be strong at:

1. Full-stack TypeScript (one language for UI + sync)
2. Relational data integrity (users, solves, contests, teams)
3. Server-side work (scraping/GraphQL must not run in the browser)
4. Fast iteration on UI (filters, heatmaps, calendars)

---

2. Stack at a glance

Layer	Choice in DevTrack	Version / notes	Primary role in this app
Language	**TypeScript**	5.x	Shared types across UI, API, adapters
Framework	**Next.js**	16 App Router	Pages, RSC, API routes, cron hooks

UI library	**React**	19	Interactive dashboard widgets
Styling	**Tailwind CSS**	v4 + CSS tokens	Design system, dark theme
Charts	**Recharts**	3.x	Activity / compare charts
Heatmap	**Custom CSS grid**	v2 component	Contribution calendar
Icons	**Lucide React**	—	Consistent icon set
Motion	**Framer Motion**	14.x	Subtle UI motion
Auth	**Auth.js (next-auth)**	v5 beta	Sessions, OAuth, credentials
Auth DB	**@auth/drizzle-adapter**	—	Persist users/accounts
Validation	**Zod**	4.x	Env + API schemas
ORM	**Drizzle ORM**	0.45	Typed SQL
DB host	**Neon Postgres**	serverless	Primary datastore
DB driver	**@neondatabase/serverless**	—	Edge/serverless-friendly queries
Client data (available)	**TanStack Query**	5.x	Optional client caching
Dates	**date-fns** / **date-fns-tz**	—	Local dates for heatmaps
Classnames	**clsx** + **tailwind-merge**	—	Safe utility merging
Passwords	**bcryptjs**	—	Credential hashing
Package manager	**pnpm**	12.x	Installs / lockfile
Tests	**Vitest** + Testing Library	—	Unit / component tests
Lint/format	**ESLint** + **Prettier**	—	Consistency
Optional email	**Resend**	env	Magic links / mail
Optional push	**VAPID**	env	Web push
Optional errors	**Sentry**	env	Production monitoring
Optional contests	**CLIST**	env	Extra contest feed

---

3. Architecture overview

```
Browser (React client widgets)
    │  filters via URL (?platforms=&range=&compare=)
    ▼
Next.js App Router
    ├── Server Components  → loadDashboardData / contests queries
    ├── Client Components  → FilterBar, Heatmap, Charts, Sheets
    └── Route Handlers     → /api/platforms/sync-all, contests refresh, cron
            │
            ▼
Server services (src/server/services)
    ├── sync.ts              → orchestrates platform sync
    ├── activity.ts          → heatmaps, streaks, recent solves
    ├── contests.ts          → contest cache reads
```

```
    └── contest-aggregator   → multi-source upcoming contests
            │
            ▼
Platform adapters (src/server/platforms)
    ├── leetcode / codeforces / atcoder / hackerrank / codechef
    └── shared PlatformAdapter interface
            │
            ▼
Neon Postgres via Drizzle
    users, platform_accounts, problems, submissions, solves,
    daily_activity, contests, teams, ...
```

Important design rules

- Third-party fetches stay on the **server**
- Heatmap/filter UX can be **client-side** over precomputed series
- Rollups (`daily_activity` , `activity_calendar` , `reported_solved_count`) protect the DB from full scans

---

4. Language & UI layer (deep dive)

4.1 TypeScript

What it is

A typed superset of JavaScript that compiles to JS. Types exist at compile time; runtime is still JavaScript.

Why DevTrack uses it

- `PlatformId` , filter state, DTOs, and adapter return types stay consistent
- Refactors (e.g. heatmap series shape) fail the build instead of production
- Env validation with Zod + `z.infer` gives runtime + compile-time safety

Key role in industry today

The default language for serious web product engineering on the JS ecosystem. If a company builds React/Node SaaS in 2025–2026, TypeScript is assumed unless there is a strong reason not to.

Alternatives

Alternative	When it wins	When it loses
JavaScript	Tiny scripts, prototypes	Large multi-dev codebases
Flow	Legacy Facebook-era code	Ecosystem almost gone for new apps
Python (Django/FastAPI) + TS frontend	Data/ML-heavy backends	Two languages, duplicated models
Go / Java / Kotlin APIs	Extreme concurrency / enterprise standards	Slower UI iteration if split poorly
Rust (WASM/backends)	Performance-critical systems	Hiring + velocity cost

How companies use it

- **Stripe, Vercel, Linear, Cal.com, many fintech frontends**: TS across app code
- **Banks / enterprises**: TS on customer portals; sometimes Java/Kotlin for core ledgers
- **Startups**: TS end-to-end to move fast with fewer runtime surprises

Efficiency tips

- Enable `strict`
- Prefer discriminated unions over loose strings (`range: '7d' | '30d' | ...`)
- Don’t leak server types into client bundles

- Generate types from Zod at boundaries

4.2 React 19

What it is

A UI library for composing interfaces from components and state.

Why DevTrack uses it

Dashboard is highly interactive: filters, heatmaps, sheets, compare mode, charts. React is the ecosystem Next.js and Recharts expect.

Key role in industry today

The dominant library for interactive product UIs (dashboards, settings, editors). Not always best for purely static marketing pages, but unmatched hiring pool and ecosystem.

Alternatives

Alternative	Pros	Cons
Vue	Gentle learning curve	Smaller enterprise hiring pool in some regions
Svelte / SvelteKit	Less boilerplate, great DX	Smaller ecosystem
Solid	Fine-grained reactivity, fast	Smaller community
Angular	Batteries-included enterprise	Heavier; different culture
Native mobile (Swift/Kotlin)	Best mobile UX	Separate apps; DevTrack is web-first

How companies use it

- **Meta**: created and drives React
- **Most SaaS**: React for app shells
- **Design-forward products (Linear-like)**: React + careful performance work

Efficiency tips

- Keep `"use client"` at leaves
- Memoize expensive derived data (`useDashboardData`)
- Prefer server-fetched props over client waterfalls when possible

---

4.3 Next.js 16 (App Router)

What it is

A React meta-framework: routing, SSR/RSC, API routes, image optimization, deployment conventions.

Why DevTrack uses it

One deployable unit:

- Marketing/landing + authenticated app
- Server Components for `loadDashboardData`
- Route handlers for sync/cron
- Suspense boundaries for `useSearchParams` filters

Key role in industry today

The default full-stack React framework, especially for products that deploy on Vercel or want App Router/RSC. It sits between “SPA only” and “separate backend microservice.”

Alternatives

Alternative	Pros	Cons
Remix	Excellent forms, nested routing	Smaller market share
Vite SPA + Nest/Fastify	Clear separation	More repos/ops, CORS, duplicated auth
Nuxt (Vue)	Great Vue full-stack	Different UI ecosystem
Redwood / Blitz-style	Opinionated full-stack	Smaller communities
Ruby on Rails / Django monolith	Mature conventions	Different language for UI interactivity

How companies use it

- **Startups**: ship MVP + marketing site together
- **Agencies**: standard delivery stack for client portals
- **Mid-size SaaS**: customer app + admin
- **Enterprises**: sometimes Next for external portals while core systems stay Java/.NET

Efficiency tips

- Personalized pages: `force-dynamic` or explicit cache tags
- Public pages: cache aggressively
- Long syncs: raise `maxDuration` or move to background jobs
- Never call LeetCode/Codeforces from the browser

4.4 Tailwind CSS v4 + tokens

What it is

Utility-first CSS. DevTrack adds semantic CSS variables (`--v2-bg` , `--v2-cf` , heat colors) in `tokens.css` .

Why DevTrack uses it

Fast UI iteration, dark theme, platform brand colors, consistent radii/spacing without CSS-in-JS runtime cost.

Key role in industry today

The leading styling approach for new SaaS UIs when teams want custom look without Material Design defaults. Often wrapped into an internal component library.

Alternatives

Alternative	Pros	Cons
CSS Modules	Explicit, local	Slower design iteration
styled-components / Emotion	Dynamic theming in JS	Runtime + SSR complexity
MUI / Chakra / Ant	Instant admin UI	Harder unique branding, bigger bundles
Vanilla Extract / Panda	Type-safe styles	Extra toolchain
Bootstrap	Familiar	Dated look if unmodified

How companies use it

- **Vercel / many modern SaaS**: Tailwind or Tailwind-like utilities
- **Enterprises**: often MUI/Ant for internal tools; Tailwind for newer customer products
- **Design systems teams**: tokens (like our `--v2-*`) are the real standard; Tailwind is one delivery mechanism

Efficiency tips

- Use tokens, not random hex

- `cn()` + `tailwind-merge` to avoid conflicting classes
- Avoid dynamically concatenating unbounded class strings

---

5. Data & backend (deep dive)

5.1 PostgreSQL (on Neon)

What it is

Relational database: tables, constraints, indexes, transactions.

Why DevTrack uses it

Data is inherently relational:

- User → many platform accounts
- Problems unique per `(platform, external_id)`
- Solves unique per `(user, problem)`
- Teams / memberships / contests cache

Key role in industry today

The default OLTP database for product applications. "When in doubt, use Postgres" is still the mainstream advice for SaaS.

Alternatives

Alternative	Pros	Cons
MySQL / MariaDB / PlanetScale	Familiar, scalable patterns	Different ecosystem tools
MongoDB	Flexible documents	Weak joins for solves/teams
DynamoDB / NoSQL	Massive scale patterns	Harder ad-hoc analytics/queries
SQLite / Turso	Simple, edge-friendly	Multi-tenant concurrency tradeoffs
Google Spanner / Cockroach	Global consistency	Cost/complexity

How companies use it

- **Startups**: Neon, Supabase, RDS, Railway
- **Scale-ups**: RDS/Aurora, Cloud SQL, self-managed Postgres
- **Analytics**: still Postgres for app DB, plus warehouse (BigQuery/Snowflake/Redshift) for heavy BI

Efficiency tips for DevTrack

- Indexes on activity `(user_id, date)`, submissions `(account_id, submitted_at)`
- Rollups: `daily_activity`, `activity_calendar`
- `reported_solved_count` when APIs truncate history
- Pooled URL for app; unpooled for migrations

5.2 Neon (serverless Postgres)

What it is

Hosted Postgres with serverless-friendly connections, branching, and separate pooled/unpooled endpoints.

Why DevTrack uses it

Fits Next.js serverless/Lambda-style deployments: many short-lived connections without running your own Postgres VM.

Key role in industry today

Make Postgres feel serverless — popular with Vercel/Next apps and small teams that don't want DBA overhead.

Alternatives

Supabase, Railway, Render Postgres, AWS RDS, Fly Postgres, self-hosted.

Company usage

Common in startups and hackathon-to-production apps. Large enterprises more often pick RDS/Aurora for compliance/networking controls — still Postgres dialect.

5.3 Drizzle ORM

What it is

TypeScript ORM/query builder with schema-as-code and SQL-shaped queries.

Why DevTrack uses it

- Schema lives in `src/server/db/schema.ts`
- Queries stay close to SQL (good for upserts, `GREATEST`, joins)
- Works cleanly with Neon serverless driver

Key role in industry today

Modern typed SQL access for TypeScript teams — competing with Prisma for "default TS ORM" mindshare. Prisma often wins on polish/migrations UX; Drizzle often wins on SQL transparency and edge/serverless fit.

Alternatives

Alternative	Pros	Cons
Prisma	Great DX, migrations, ecosystem	Heavier client historically
Kysely	Extremely typed SQL	More manual
TypeORM / Sequelize	Mature	Older patterns
Raw `postgres.js` / `pg`	Max control	No schema layer

How companies use it

- Startups: Prisma or Drizzle
- Enterprises: often Hibernate/jOOQ/Entity Framework on non-JS stacks; JS shops still Prisma-heavy

Efficiency tips

- Select only needed columns
- Batch inserts (sync already batches submissions)
- Prefer migrations in prod over interactive `db push`

---

6. Auth, validation, security

6.1 Auth.js (NextAuth v5) + Drizzle adapter

Why DevTrack uses it

Sessions for the app, OAuth providers (GitHub/Google), credentials option, DB-backed users via adapter — without building crypto/session storage from scratch.

Key role in industry today

Auth for Next apps when you want ownership of user tables and lower vendor cost. Hosted auth (Clerk/Auth0/WorkOS) wins when you need polished UI, organizations, or enterprise SSO fast.

Alternatives

Alternative	Best for
Clerk	Fast SaaS UX, orgs, prebuilt components
Auth0 / Okta / WorkOS	Enterprise SSO/SAML
Supabase Auth	Supabase-centric apps
Lucia / custom sessions	Minimal custom stacks
Firebase Auth	Mobile + Google ecosystem

Company usage

- Indie/Next products: Auth.js or Clerk
- B2B enterprise sales: Okta/Auth0/WorkOS almost required
- Consumer apps: Cognito/Firebase/Clerk common

Security practices in DevTrack

- Validate env with Zod (`AUTH_SECRET` min length)
- Protect cron with `CRON_SECRET`
- Rate-limit sync-all
- Never commit `.env.local` / session cookies (`LEETCODE_SESSION`)

6.2 Zod

Why

Runtime validation for environment variables and untrusted inputs.

Key role today

De-facto TS validation library at API boundaries (often with tRPC, Next route handlers, form resolvers).

Alternatives

Valibot, ArkType, Yup, Joi, Superstruct.

Company usage

Widespread in Node/TS services; schema validation is considered basic hygiene.

---

7. Charts, motion, design system

7.1 Recharts

Role in DevTrack: activity bar charts; compare mode multi-series.
Role in industry: in-app product charts (not a replacement for Tableau/Looker).
Alternatives: Chart.js, Nivo, Visx, ECharts, Observable Plot.
Companies: admin dashboards, analytics widgets inside SaaS. Heavy observability uses Grafana/custom canvas.

7.2 Custom ActivityHeatmap (CSS grid)

Why not Recharts: thousands of day cells are cheaper as div/button grids than SVG series.
Industry parallel: GitHub contribution graph, LeetCode calendar — almost always custom DOM/canvas, not general chart libs.
DevTrack extras: per-platform hover breakdown when multiple platforms are selected.

7.3 Lucide + Framer Motion

Lucide: consistent open icon set (alternative: Heroicons, Phosphor, Font Awesome).
Framer Motion: intentional micro-interactions (alternative: CSS transitions only, React Spring).

Industry: design-quality products invest in motion carefully; enterprise admin tools often skip it.

7.4 Platform design tokens (`PLATFORM_META`)

Central map of label/short/color for chips, badges, charts, tooltips.
Industry parallel: design tokens in Figma → CSS variables → components (Salesforce Lightning, Material tokens, etc.).

---

8. DevTrack domain systems

8.1 Platform adapters

Files: `src/server/platforms/*` , `types.ts`
Pattern: Strategy/Adapter — each site implements `fetchProfile` , `fetchSubmissions` , contests, optional `fetchActivityCalendar` .

Why
External APIs differ wildly (CF REST vs LC GraphQL vs CC HTML). Adapters quarantine that mess.

Industry equivalent

Industry system	Same idea
Segment sources	Normalize vendor events
Plaid	Normalize bank APIs
Merge.dev	Unified HR/ATS APIs
Zapier connectors	Per-app auth + actions
GitHub Apps / webhooks	Integrate many repos/orgs

Efficiency
Backoff, incremental `since` , calendar cache, reported totals when lists are truncated, optional session for full LC history.

Limitation (important)
LeetCode does **not** publicly expose a full solved-problem list. Public APIs ≈ recent 20 + calendar + submitStats. Full list needs user session cookie (`LEETCODE_SESSION`) or official export.

8.2 Sync pipeline

File: `sync.ts`
Flow: create sync run → fetch submissions → upsert problems/submissions/solves → rollup daily activity → fetch calendar → update profile/reported counts → mark success/error.

Industry equivalent: ETL/ELT micro-batches; “sync jobs” in HubSpot/Salesforce connectors; billing usage importers.

Make it more efficient
Move to a queue (Inngest/Trigger.dev/BullMQ), show job progress, checkpoint pages, idempotent upserts (already mostly true).

8.3 Contests aggregation

Files: `contests.ts` , `contest-aggregator.ts` , refresh API, cron
Sources: platform adapters + ContestRadar/CLIST-style feeds
Store: Postgres contests table, serve UI from cache

Industry equivalent: flight/hotel aggregators, sports odds feeds, CI status dashboards — cache externally fetched schedules.

8.4 Dashboard filters & compare mode

Files: `useDashboardFilters`, `useDashboardData`, `FilterBar`, Overview screen
State: URL search params (shareable, back/forward safe)
Behavior: one filter state drives stats, heatmap, chart, recent feed; compare splits series

Industry equivalent: analytics product filter bars (Mixpanel/Amplitude UI patterns), cloud console resource filters.

---

9. Tooling, CI, quality

Tool	Why here	Industry role today	Alternatives
pnpm	Fast, strict, less disk waste	Modern JS package manager of choice in many monorepos	npm, yarn, bun
Vitest	Fast TS tests	Jest successor for Vite-world apps	Jest, node:test
Testing Library	Tests behave like users	Standard React testing approach	Cypress/Playwright for E2E
ESLint	Catch bugs/style	Still dominant linter	Biome, oxlint
Prettier	Format wars end	Default formatter	Biome format
tsx	Run TS scripts	Local scripting	ts-node

Company practice
Serious teams run typecheck + lint + tests on every PR. E2E (Playwright) is added when user flows (auth, billing, sync) become critical.

---

10. Optional services

Service	DevTrack use	Industry role	Alternatives
Resend	Email	Transactional email API	SES, Postmark, SendGrid
VAPID web push	Reminders	Browser push without native apps	FCM, OneSignal
Sentry	Errors	Application monitoring	Datadog, Highlight, Bugsnag
CLIST	Contests	Niche contest meta-API	Manual/platform APIs
LEETCODE_SESSION	Full LC history	User-authorized scrape/session sync	Official exports only

---

11. Key role of each technology today

Use this as a "what is this technology *for* in 2025–2026?" cheat sheet.

Platform & language

Tech	One-sentence key role today
TypeScript	The contract language of web product teams — fewer production type bugs, better refactors
JavaScript runtime (Node)	The server runtime that hosts Next API routes and sync jobs
React	The UI engine for stateful product interfaces
Next.js	The opinionated way to ship React as a full product (app + API + SSR)
Tailwind	The fastest mainstream way to build custom-looking UI systems

Data

Tech	One-sentence key role today
PostgreSQL	The trustworthy system of record for app data
Neon	Postgres consumed as a cloud/serverless dependency
Drizzle	Typed gateway between TS code and SQL
Rollup tables / caches	How products make dashboards fast without scanning raw events live

Trust & correctness

Tech	One-sentence key role today
Auth.js / Clerk / Okta	Prove who the user is; gate data
Zod	Prove untrusted input matches the shape you coded for
Rate limits / cron secrets	Prove bots and strangers can't burn your sync quota

Product experience

Tech	One-sentence key role today
Recharts	Small in-app analytics, not enterprise BI
Custom heatmaps	Habit/progress storytelling (GitHub/LeetCode pattern)
URL-driven filters	Shareable, bookmarkable analysis state

Integrations

Tech	One-sentence key role today
Adapters + sync jobs	The backbone of "connected SaaS" — bring external truth into your DB
Queues (future)	How companies keep HTTP requests fast while sync runs long

---

12. What companies are doing with this tech

12.1 Developer tooling & product SaaS (closest cousins)

Company / type	What they do with overlapping tech	Relevance to DevTrack
Vercel	Builds/hosts Next.js; TS/React culture	Our deploy & framework home turf
Linear	Extremely polished React app; performance obsession	UX bar for dashboards/filters
Cal.com	Open-source Next + DB + auth style SaaS	Similar full-stack shape
Resend	Email API + modern Next marketing/docs	Optional mail provider model
GitHub	Integrations/webhooks/apps at massive scale	Adapter mindset for external systems
Stripe	TS-heavy docs/SDKs; webhook integrations	"Normalize external events" pattern

12.2 EdTech / coding career products (same domain)

Company	What they do	What DevTrack borrows conceptually
LeetCode	Practice, heatmap, contests, ratings	Heatmap UX, calendar, stats
Codeforces	Contests + rating + API	Official API sync
AtCoder	Contests + rating	Kenkoooo/API style history
HackerRank	Practice + hiring	Company hiring dashboards
CodeChef	Practice + contests	Contests aggregation
Interview prep startups	Track progress across resources	Multi-source progress narrative

12.3 Integration platforms (same architecture, different domain)

Company	Business	Shared architecture idea
Plaid	Bank data aggregation	Fragile external APIs → normalized ledger
Segment	Customer data pipeline	Source adapters → warehouse
Zapier / Make	Workflow automation	Connectors + retries + rate limits
Merge	Unified HR/ATS API	One interface over many vendors
n8n	Open-source automation	Self-hosted connector model

If someone asks "what is DevTrack like architecturally?", a strong answer is:
"A vertical Plaid/Segment-style sync for coding platforms, with a Linear-like progress UI."

12.4 Big tech & enterprise (building blocks at scale)

Org type	How the building blocks appear	What they add beyond our stack
Meta	React ecosystem leadership	Internal infra at insane scale
Google	Many consoles; analytics warehouses	BigQuery, Spanner, IAM
Amazon	Heterogeneous services	Aurora, IAM, multi-region
Fintech / banks	TS frontends + strict APIs	SSO, audits, change management
Healthtech	Similar web stacks	HIPAA, BAA, stricter hosting

They often use **the same product ideas** (dashboards, sync jobs, Postgres) but wrap them in compliance, multi-region, and dedicated platform teams.

12.5 Agencies & internal tools

Who	What they do with Next/Tailwind/Postgres
Product agencies	Ship client dashboards quickly
Internal eng tools teams	Hiring funnels, learning trackers, ops consoles
University clubs / bootcamps	Contest calendars + progress leaderboards

DevTrack is a strong portfolio example for these roles because it shows **UI + data modeling + integrations**.

---

13. Company role map (who uses what for what)

Job / team	Technologies they care about	How DevTrack demonstrates it
Frontend engineer	React, Next, Tailwind, accessibility, charts	Dashboard filters, heatmap tooltips, contests calendar
Full-stack engineer	Next RSC + route handlers + DB	`loadDashboardData` + sync APIs
Backend engineer	Postgres, Drizzle, jobs, rate limits	Adapters, upserts, rollups
Integration engineer	External APIs, retries, normalization	Platform adapters
Design systems / UI eng	Tokens, components, dark mode	`tokens.css`, `PLATFORM_META`, v2 components
DevOps / platform	CI, env management, monitoring	Zod env, optional Sentry, cron secrets
Data / analytics eng	Warehouses, rollups	`daily_activity` as a mini analytics table
Security eng	Auth, secrets, least privilege	Auth.js, cron auth, no secret commits
Product manager	UX of progress, retention loops	Streaks/heatmaps/compare filters
Engineering manager	Maintainability, hiring stack familiarity	Mainstream TS/Next/Postgres choices

---

14. Efficiency playbook (current + roadmap)

14.1 What we already do well

- Server-only third-party fetches
- Incremental sync window via `lastSyncedAt`
- Heatmap from calendars/rollups, not full submission scans every render
- URL filters without full page reload scroll jumps
- Batch submission inserts
- Reported LeetCode totals when history is truncated
- Contest DB cache + cron/manual refresh
- Per-platform heatmap hover breakdown when multiple platforms selected

14.2 Highest-impact next upgrades

Priority	Upgrade	Why
P0	Background job queue for sync	HTTP timeouts / poor UX on full LC backfill
P0	Checkpointed LC session sync	Don't re-pull entire history every time
P1	Redis/Upstash cache for contests + public profiles	Lower DB/API load
P1	Playwright E2E for auth + sync happy path	Catch breaks early
P2	Materialized leaderboard tables	Teams at scale
P2	Read replica / pooled tuning	More concurrent users
P3	Warehouse export (optional)	Serious org analytics

14.3 Frontend performance checklist

- Keep heatmap DOM reasonable (cap years; denser cells for long ranges)
- Memoize derived filter data
- Code-split heavy chart paths if bundle grows
- Prefer CSS transitions when Framer isn't needed

14.4 Backend performance checklist

- Index hot columns
- Avoid N+1 account loops where batching works
- Respect platform rate limits (shared sleeps)
- Separate “stats totals” from “full problem list availability”

---

15. When to keep vs replace a choice

Choice	Keep when…	Replace when…
Next.js	One app, RSC, fast product iteration	You need polyglot microservices-first architecture
Drizzle + Neon	TS SQL + serverless Postgres	Company standard is Prisma/RDS only, or global Spanner requirement
Auth.js	Cost control, own user tables	Enterprise SSO/SAML is a sales blocker → WorkOS/Auth0
Tailwind	Custom branded UI	Mandatory Material/Ant corporate kit
Recharts	Simple in-app charts	You need streaming telemetry dashboards → ECharts/Grafana
Custom heatmap	GitHub/LC-style calendars	Millions of cells → canvas/WebGL
In-request sync	Low users, demos	Sync > 30–60s regularly → queue
Adapter scraping	No official full API	Vendor provides stable official bulk export/API

---

16. Interview / portfolio talking points

If you explain DevTrack to a company, these are strong, accurate claims:

1. **“Full-stack TypeScript with Next App Router”** — server components for personalized data, client components for interactive filters.
2. **“Integration architecture”** — platform adapters normalize Codeforces/LeetCode/AtCoder into one schema (mini Segment/Plaid pattern).
3. **“Analytics UX without a warehouse”** — rollup tables + calendars power heatmaps/streaks efficiently.
4. **“URL-driven dashboard state”** — shareable filters/compare mode.
5. **“Honest API constraints”** — LeetCode public APIs truncate history; we use submitStats + optional session rather than fake completeness.
6. **“Product polish”** — design tokens, dark theme, per-platform heatmap breakdowns, contests calendar.

---

17. Related docs

- `docs/SPEC.md` — product & data model specification
- `docs/IMPLEMENTATION_GUIDE.md` — implementation notes
- `.env.example` — environment variables and optional providers
- `src/server/platforms/types.ts` — adapter contract
- `src/v2/hooks/useDashboardFilters.ts` — filter state machine

---

18. Appendices

Appendix A — Dependency map (mental model)

```
pnpm
 └─ Next.js
     ├─ React + Tailwind + Lucide (+ Framer / Recharts on client)
     ├─ Auth.js ── Drizzle adapter ── Neon Postgres
     ├─ Zod (env/input)
```

```
    └ Server services
        ├ Drizzle queries
        └ Platform adapters (HTTP/GraphQL to coding sites)
```

Appendix B — Data truth sources by platform (practical)

Platform	Best official/public signal	Weak spot
Codeforces	`user.status` API	Huge histories need pagination caps
LeetCode	submitStats + calendar; session for full list	Public recent list truncated (~20)
AtCoder	Kenkoooo / site JSON	Windowing for huge histories
HackerRank	public REST recent challenges	Incomplete for some profiles
CodeChef	profile/contest pages	Submissions limited / brittle HTML

Appendix C — Where things live in the repo

Concern	Path
UI screens	`src/v2/screens/`
Design tokens / platforms	`src/v2/design/`
Dashboard filters	`src/v2/hooks/`, `src/v2/components/dashboard/`
DB schema	`src/server/db/schema.ts`
Sync	`src/server/services/sync.ts`
Heatmap queries	`src/server/services/activity.ts`
Platform adapters	`src/server/platforms/`
Env validation	`src/env.ts`

Document version: full industry + stack guide for the current DevTrack codebase (Next.js 16, React 19, Tailwind 4, Drizzle, Neon, Auth.js v5, Recharts, Vitest, pnpm).